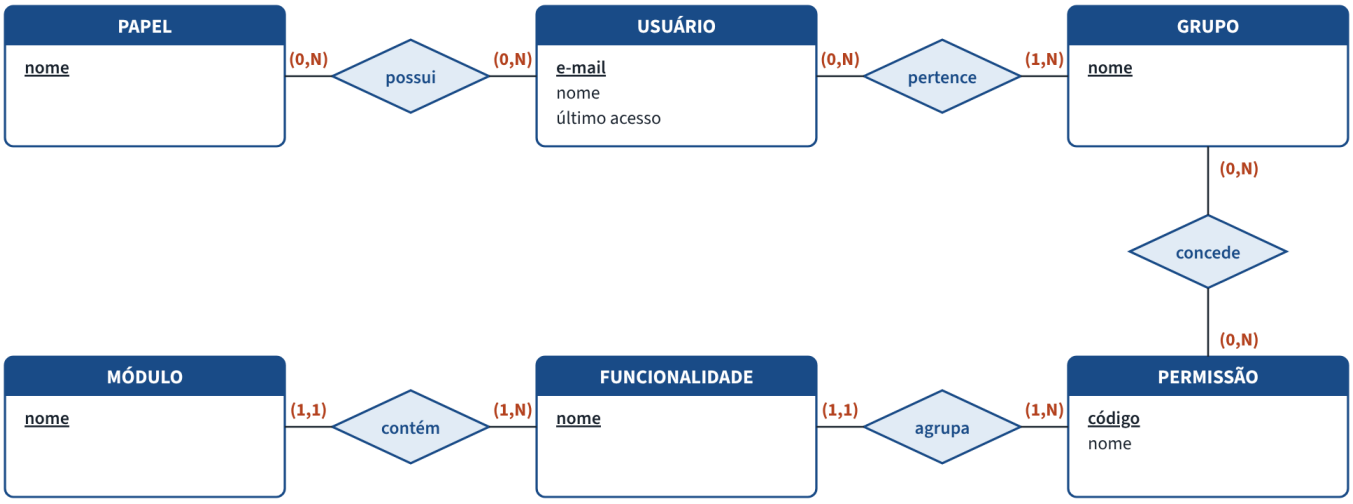
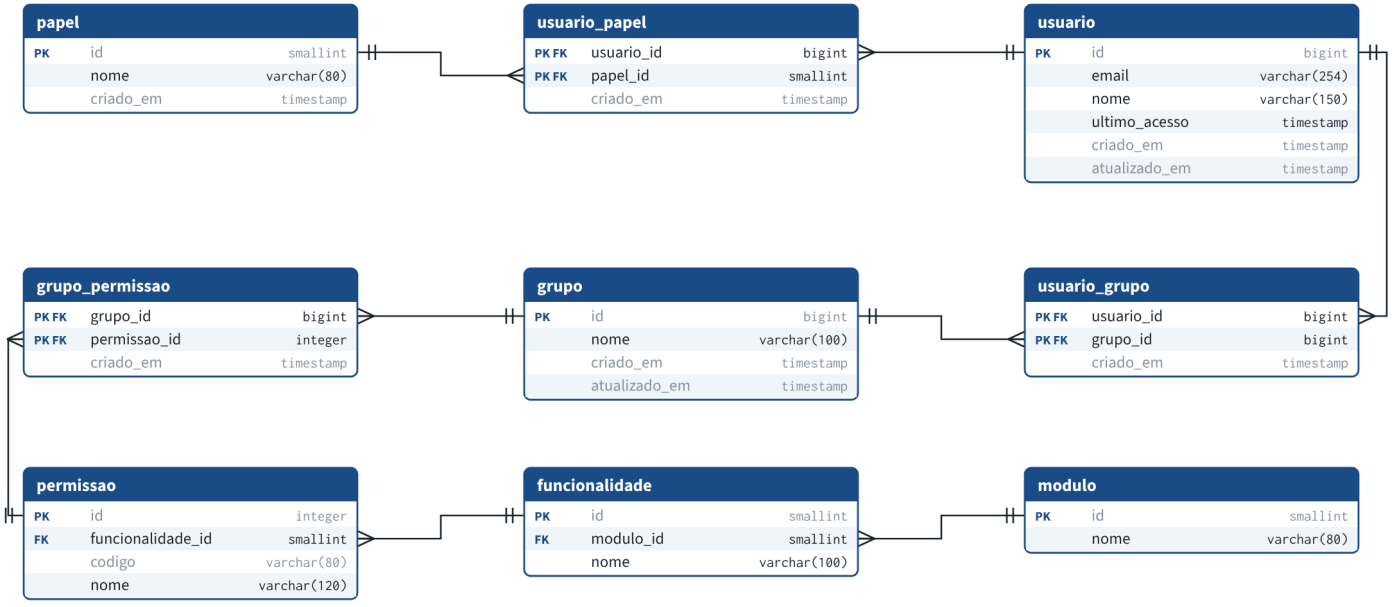
Módulo de Controle de Acessos — Modelo Físico e Consultas

Banco de Dados · Aluno(a): _____ · PostgreSQL

1. Modelo conceitual



2. Modelo físico



PK = chave primária · FK = chave estrangeira · atributos de negócio em preto, atributos técnicos em cinza (chaves substitutas, código e datas de controle). As relações N:N viram as tabelas associativas *usuario_grupo*, *usuario_papel* e *grupo_permissao*. A senha não fica no banco: o login é feito pelo Active Directory.

Script de criação (tipos e restrições)

```sql
CREATE TABLE usuario (
  id             bigint       GENERATED ALWAYS AS IDENTITY,
  email          varchar(254) NOT NULL,
  nome           varchar(150),    -- vem do AD
  ultimo_acesso  timestamp,       -- NULL = nunca acessou
  criado_em      timestamp    NOT NULL DEFAULT now(),
  atualizado_em  timestamp    NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario PRIMARY KEY (id),
  CONSTRAINT uq_usuario_email UNIQUE (email),
  CONSTRAINT ck_usuario_email
    CHECK (email = lower(email) AND email LIKE '%_@_%._%')
);
```

```sql
CREATE TABLE papel (
  id         smallint     GENERATED ALWAYS AS IDENTITY,
  nome       varchar(80)  NOT NULL,
  criado_em  timestamp    NOT NULL DEFAULT now(),
  CONSTRAINT pk_papel PRIMARY KEY (id),
  CONSTRAINT uq_papel_nome UNIQUE (nome)
);
```

```sql
CREATE TABLE grupo (
  id             bigint       GENERATED ALWAYS AS IDENTITY,
  nome           varchar(100) NOT NULL,
  criado_em      timestamp    NOT NULL DEFAULT now(),
  atualizado_em  timestamp    NOT NULL DEFAULT now(),
  CONSTRAINT pk_grupo PRIMARY KEY (id),
  CONSTRAINT uq_grupo_nome UNIQUE (nome)
);
```

```sql
CREATE TABLE modulo (
  id    smallint    GENERATED ALWAYS AS IDENTITY,
  nome  varchar(80) NOT NULL,
  CONSTRAINT pk_modulo PRIMARY KEY (id),
  CONSTRAINT uq_modulo_nome UNIQUE (nome)
);
```

```sql
CREATE TABLE funcionalidade (
  id         smallint     GENERATED ALWAYS AS IDENTITY,
  modulo_id  smallint     NOT NULL,
  nome       varchar(100) NOT NULL,
  CONSTRAINT pk_funcionalidade PRIMARY KEY (id),
  CONSTRAINT uq_funcionalidade_nome UNIQUE (modulo_id, nome),
  CONSTRAINT fk_funcionalidade_modulo FOREIGN KEY (modulo_id)
    REFERENCES modulo (id) ON DELETE RESTRICT
);
```

```sql
CREATE TABLE permissao (
  id                integer      GENERATED ALWAYS AS IDENTITY,
  funcionalidade_id  smallint     NOT NULL,
  codigo            varchar(80)  NOT NULL,  -- usado no código
  nome              varchar(120) NOT NULL,  -- texto do toggle
  CONSTRAINT pk_permissao PRIMARY KEY (id),
  CONSTRAINT uq_permissao_codigo UNIQUE (codigo),
  CONSTRAINT uq_permissao_nome
    UNIQUE (funcionalidade_id, nome),
  CONSTRAINT fk_permissao_funcionalidade
    FOREIGN KEY (funcionalidade_id)
    REFERENCES funcionalidade (id) ON DELETE RESTRICT
);
```

```sql
-- usuario N:N grupo
CREATE TABLE usuario_grupo (
  usuario_id  bigint    NOT NULL,
  grupo_id    bigint    NOT NULL,
  criado_em   timestamp NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario_grupo
    PRIMARY KEY (usuario_id, grupo_id),
  CONSTRAINT fk_usuario_grupo_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuario (id) ON DELETE CASCADE,
  CONSTRAINT fk_usuario_grupo_grupo FOREIGN KEY (grupo_id)
    REFERENCES grupo (id) ON DELETE CASCADE
);
```

```sql
-- usuario N:N papel
CREATE TABLE usuario_papel (
  usuario_id  bigint    NOT NULL,
  papel_id    smallint  NOT NULL,
  criado_em   timestamp NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario_papel
    PRIMARY KEY (usuario_id, papel_id),
  CONSTRAINT fk_usuario_papel_usuario FOREIGN KEY (usuario_id)
    REFERENCES usuario (id) ON DELETE CASCADE,
  CONSTRAINT fk_usuario_papel_papel FOREIGN KEY (papel_id)
    REFERENCES papel (id) ON DELETE RESTRICT
);
```

```sql
-- grupo N:N permissao (linha = toggle habilitado)
CREATE TABLE grupo_permissao (
  grupo_id      bigint    NOT NULL,
  permissao_id  integer   NOT NULL,
  criado_em     timestamp NOT NULL DEFAULT now(),
  CONSTRAINT pk_grupo_permissao
    PRIMARY KEY (grupo_id, permissao_id),
  CONSTRAINT fk_grupo_permissao_grupo FOREIGN KEY (grupo_id)
    REFERENCES grupo (id) ON DELETE CASCADE,
  CONSTRAINT fk_grupo_permissao_permissao
    FOREIGN KEY (permissao_id)
    REFERENCES permissao (id) ON DELETE CASCADE
);
```

3. Consultas SQL das interfaces

Login com Active Directory

1 E-mail autenticado no AD está cadastrado?

```sql
SELECT id, nome
FROM usuario
WHERE email = 'adailton@positivo.com.br';
```

2 Atualizar nome (vindo do AD) e último acesso

```sql
UPDATE usuario
SET nome = 'Adailton Araújo',
    ultimo_acesso = now(),
    atualizado_em = now()
WHERE id = 1;
```

3 Permissões do usuário logado (de todos os seus grupos)

```sql
SELECT DISTINCT p.codigo, p.nome
FROM usuario_grupo ug
JOIN grupo_permissao gp ON gp.grupo_id = ug.grupo_id
JOIN permissao p ON p.id = gp.permissao_id
WHERE ug.usuario_id = 1
ORDER BY p.codigo;
```

Permissões - aba Usuários

4 Listar usuários com busca

```sql
SELECT u.id,
       u.nome,
       u.email,
       (SELECT string_agg(g.nome, ', ' ORDER BY g.nome)
          FROM usuario_grupo ug JOIN grupo g ON g.id = ug.grupo_id
         WHERE ug.usuario_id = u.id) AS grupos,
       (SELECT string_agg(p.nome, ', ' ORDER BY p.nome)
          FROM usuario_papel up JOIN papel p ON p.id = up.papel_id
         WHERE up.usuario_id = u.id) AS papeis,
       u.ultimo_acesso
FROM usuario u
WHERE u.nome ILIKE '%positivo%' OR u.email ILIKE '%positivo%'
ORDER BY u.nome;
```

5 Opções do campo Grupo

```sql
SELECT id, nome FROM grupo ORDER BY nome;
```

6 Opções do campo Papel

```sql
SELECT id, nome FROM papel ORDER BY nome;
```

7 Adicionar usuário

```sql
BEGIN;
INSERT INTO usuario (email) VALUES ('maria.lima@positivo.com.br');

INSERT INTO usuario_grupo (usuario_id, grupo_id)
SELECT u.id, g.id
FROM usuario u, grupo g
WHERE u.email = 'maria.lima@positivo.com.br' AND g.id IN (1, 4);

INSERT INTO usuario_papel (usuario_id, papel_id)
SELECT u.id, p.id
FROM usuario u, papel p
WHERE u.email = 'maria.lima@positivo.com.br' AND p.id IN (2);
COMMIT;
```

8 Carregar usuário para edição

```sql
SELECT email FROM usuario WHERE id = 6;
SELECT grupo_id FROM usuario_grupo WHERE usuario_id = 6;
SELECT papel_id FROM usuario_papel WHERE usuario_id = 6;
```

9 Salvar edição do usuário

```sql
BEGIN;
UPDATE usuario
SET email = 'maria.silva@positivo.com.br',
    atualizado_em = now()
WHERE id = 6;

DELETE FROM usuario_grupo WHERE usuario_id = 6;
INSERT INTO usuario_grupo (usuario_id, grupo_id)
VALUES (6, 1), (6, 2);

DELETE FROM usuario_papel WHERE usuario_id = 6;
INSERT INTO usuario_papel (usuario_id, papel_id)
VALUES (6, 1);
COMMIT;
```

10 Excluir usuário (vínculos saem por CASCADE)

```sql
DELETE FROM usuario WHERE id = 6;
```

Permissões - aba Grupos

11 Listar grupos com busca

```sql
SELECT g.id,
       g.nome,
       (SELECT count(*) FROM grupo_permissao gp
         WHERE gp.grupo_id = g.id) AS permissoes,
       (SELECT count(*) FROM usuario_grupo ug
         WHERE ug.grupo_id = g.id) AS usuarios
FROM grupo g
WHERE g.nome ILIKE '%editais%'
ORDER BY g.nome;
```

12 Usuários de um grupo

```sql
SELECT u.nome, u.email
FROM usuario_grupo ug
JOIN usuario u ON u.id = ug.usuario_id
WHERE ug.grupo_id = 4
ORDER BY u.nome;
```

Adicionar / Editar grupo

13 Aba Permissões: toggles do grupo

```sql
SELECT m.nome AS modulo,
       f.nome AS funcionalidade,
       p.id   AS permissao_id,
       p.nome AS permissao,
       (gp.grupo_id IS NOT NULL) AS habilitada
FROM permissao p
JOIN funcionalidade f ON f.id = p.funcionalidade_id
JOIN modulo m ON m.id = f.modulo_id
LEFT JOIN grupo_permissao gp
       ON gp.permissao_id = p.id AND gp.grupo_id = 4
ORDER BY m.id, f.id, p.id;
```

14 Aba Usuários: checkbox de cada usuário

```sql
SELECT u.id,
       u.nome,
       u.email,
       EXISTS (SELECT 1 FROM usuario_grupo x
               WHERE x.usuario_id = u.id
                 AND x.grupo_id = 4) AS marcado,
       (SELECT string_agg(g.nome, ', ' ORDER BY g.nome)
          FROM usuario_grupo ug JOIN grupo g ON g.id = ug.grupo_id
         WHERE ug.usuario_id = u.id) AS grupos
FROM usuario u
ORDER BY u.nome;
```

15 Adicionar grupo

```sql
BEGIN;
INSERT INTO grupo (nome) VALUES ('Análise de propostas');

INSERT INTO grupo_permissao (grupo_id, permissao_id)
SELECT g.id, p.id
FROM grupo g, permissao p
WHERE g.nome = 'Análise de propostas' AND p.id IN (6, 7, 9);

INSERT INTO usuario_grupo (usuario_id, grupo_id)
SELECT u.id, g.id
FROM usuario u, grupo g
WHERE g.nome = 'Análise de propostas' AND u.id IN (2, 4);
COMMIT;
```

16 Salvar edição do grupo

```sql
BEGIN;
UPDATE grupo
SET nome = 'Análise de propostas e editais',
    atualizado_em = now()
WHERE id = 6;

DELETE FROM grupo_permissao WHERE grupo_id = 6;
INSERT INTO grupo_permissao (grupo_id, permissao_id)
VALUES (6, 6), (6, 7), (6, 9), (6, 11);

DELETE FROM usuario_grupo WHERE grupo_id = 6;
INSERT INTO usuario_grupo (usuario_id, grupo_id)
VALUES (2, 6), (4, 6);
COMMIT;
```

17 Excluir grupo (vínculos saem por CASCADE)

```sql
DELETE FROM grupo WHERE id = 6;
```